

Lab 01 - Vision-System Needs Analysis and Generative Pipeline Specification

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 01: Generative AI Fundamentals for Image and Video Creation Outcome: ELO1 | In-class time: facilitated within LU1 classroom facilitation (no separate practical minutes)

Competency	Statement
Ability A1	Identify the needs of vision systems technology in industrial applications
Knowledge K1	Vision system concepts
Knowledge K2	Business applications of vision systems

Scenario

NorthQuay Retail Group runs 41 stores and an online catalogue. Six departments have each asked for 'AI images'. You are the vision-systems analyst: score the six candidate use cases against measurable need criteria, reject the ones that do not need a vision system, and specify the five-stage pipeline for the two that survive.

What you will produce

A completed needs-analysis scoring matrix, a written rejection rationale, and a labelled five-stage pipeline specification for the two selected use cases.

Tools: Spreadsheet or text editor | Python 3 (optional scorer) | no paid service required

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3 (optional - the scorer script is provided).

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Read the brief and the six candidate cases
2. Score each case on the six need criteria
3. Reject the cases that fail the need test

- Specify the 5-stage pipeline for the survivors
- Record the decision rationale as evidence

Files in this folder

Path	What it is
reference/nq-storefront-reference.png	Synthetic reference frame representing a NorthQuay store-shelf camera view (deterministically rendered with OpenCV - labelled SIMULATED, not a photograph and not generative-model output)
data/candidate-use-cases.csv	Six candidate use cases with volume, latency tolerance, accuracy tolerance, privacy class, current manual cost and image-supply notes
data/need-criteria.json	The six weighted need criteria and the accept/reject threshold
brief/northquay-brief.md	The one-page client brief with constraints and the deliverable list
score_needs.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

- Work inside this lab folder. Create out/ with your file manager. Python is optional: use the manual route below if unavailable. Create out/needs-scores.csv with headers case_id,name,volume,repeatibility,latency_class,accuracy_tolerance,privacy_risk,cost_gap,weighted_total,verdict. Enter six rows. For each row calculate the weighted total with SUMPRODUCT(score cells, weight cells) using weights from need-criteria.json; do not include non-score columns.
- Before scoring, create out/eligibility.csv with columns case_id,vision_needed,cost_ok,turnaround_ok,privacy_ok,evidence. Record PASS, FAIL or UNKNOWN in every gate. Verify that visual inputs are necessary and compare projected per-asset cost, turnaround and permitted processing against the brief. UNKNOWN and FAIL mean HOLD/REJECT regardless of score; obtain missing evidence rather than inventing it. Annotate the final manual verdict with these gates. The Python scorer creates a preliminary feasibility baseline only; it never overrides these gates.
- Open brief/northquay-brief.md and read the client constraints. Note the three hard constraints stated there: the S\$ per-asset ceiling, the maximum acceptable turnaround, and the customer-image privacy rule.
- Open data/candidate-use-cases.csv. Confirm it has 6 data rows and all columns listed in the header (the scorer needs nine fields; image_supply may provide additional narrative evidence). Read every row before scoring anything.

```
python3 -c "import csv;r=list(csv.DictReader(open('data/candidate-use-cases.csv')));print(len(r), list(r[0]))"
```

- Open data/need-criteria.json. The six criteria are volume, repeatability, latency_class, accuracy_tolerance, privacy_risk and cost_gap. Each is scored 0-5 and carries a weight; the accept threshold is stated in the same file.

6. Score criterion 1 - VOLUME. A vision system earns its build cost through throughput. Score 5 where the task repeats at least 500 times a month, 3 for 50 to under 500, 1 for under 50. Write your score in your matrix, not in the CSV.
7. Score criterion 2 - REPEATABILITY. Score 5 where the visual task is the same every time (same framing, same object class, same output format) and 1 where every instance is bespoke. Low repeatability can increase implementation cost and reduce whether a vision project pays back.
8. Score criterion 3 - LATENCY CLASS. Score 5 for batch/overnight (easy), 3 for interactive seconds, 1 for hard real-time under 100 ms. This score is inverted against difficulty on purpose: a high score means the latency requirement does not constrain your architecture.
9. Score criterion 4 - ACCURACY TOLERANCE. Score 5 where a human reviews every output before it ships (generous tolerance) and 1 where a wrong output reaches a customer or a safety decision unreviewed.
10. Score criterion 5 - PRIVACY RISK, inverted. Score 5 where no identifiable person or customer data is involved and 1 where customer faces or personal data are unavoidable. Cross-check against the `privacy_class` column in the CSV.
11. Score criterion 6 - COST GAP. Compute $(\text{current_manual_cost_sgd_per_month} - \text{estimated_automated_cost_sgd_per_month}) / \text{current_manual_cost_sgd_per_month}$. Score 5 at or above 0.6, 3 for 0.3 to below 0.6, 1 below 0.3.
12. Compute the weighted total for each case: $\text{total} = \sum(\text{score}_i * \text{weight}_i)$. Do this by hand for at least one case so you can explain the arithmetic, optionally run the provided scorer for a deterministic baseline; it does not read your manual scores. Reconcile every difference in writing.

```
python3 score_needs.py --csv data/candidate-use-cases.csv --criteria data/need-criteria.json --out out/baseline-scores.csv
```

13. Compare each weighted total against the accept threshold in `data/need-criteria.json`. The supplied baseline is expected to select two cases. If your justified scores differ, compare with the baseline and document the evidence; do not change scores to force a count.
14. Write one sentence of rejection rationale for each rejected or held case. Name the criterion that failed and the number. 'Low volume' is not a rationale; 'volume 1/5 - 22 instances a month against a 500-instance classroom scoring boundary (not a measured payback point)' is.
15. For each of the selected eligible cases, specify the five-stage pipeline. Stage 1 SENSE: name the image source (existing catalogue asset, store camera, or a text prompt) and its available resolution and rate; for text input mark pixel fields not applicable and specify intended output dimensions.
16. Stage 2 REPRESENT: state the colour space and the working resolution, and where the asset is stored. Generate a candidate before its output-quality review. Stage 3 ANALYSE: name the specific measurement that will gate quality - for example an HSV histogram distance against the brand reference, or detector recall measured against an annotated validation set, never an unlabelled live frame.
17. Stage 4 DECIDE: state the numeric threshold and what happens on each side of it (auto-publish, human review queue, reject and regenerate). Mark each proposed threshold provisional until calibrated on representative labelled examples; policy rules may be categorical.
18. Stage 5 ACT: publish or route the reviewed artefact; on failure loop back to generation. State its format and aspect ratios, and who consumes it. Note explicitly whether the stage is analytical (measure and report) or generative (create a new asset).
19. Open `reference/nq-storefront-reference.png` and confirm the label banner reads SIMULATED. Use it as the stand-in shelf frame in your Stage 1 description and record in your write-up that it is a synthetic classroom asset, not a photograph and not a generative-model output.
20. Compare `out/baseline-scores.csv` if you ran Python; keep the manual gated decisions in `out/needs-scores.csv`. Save your completed matrix as `out/needs-analysis.md` (or `.csv`) together with the selected pipeline specifications, and

complete the evidence checklist below.

Verify

Your out/needs-scores.csv shows all six cases with explicit eligibility gates and weighted scores against the accept threshold, every rejected or held case has a numbered rationale naming the failing criterion, and each surviving case has all five pipeline stages filled in with a named measurement and a numeric decision threshold.

Expected outputs

- out/needs-scores.csv - 6 rows, each with six criterion scores, a weighted total and an ACCEPT/REJECT verdict.
- Final ACCEPT verdicts require all four eligibility gates to PASS; explain any difference from the two-case baseline.
- out/needs-analysis.md - rejection or hold rationales, each naming a criterion and its score.
- Five-stage pipeline specifications for the selected eligible cases, each with a named Stage 3 measurement and a numeric Stage 4 threshold.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- [] Screenshot or paste of out/needs-scores.csv showing the weighted totals, plus out/eligibility.csv. A manually completed equivalent CSV is accepted.
- [] The accepted case IDs, their totals and all four eligibility gates; explain differences from the baseline.
- [] Rejection or hold rationales, one per rejected case, each citing a criterion score.
- [] Completed pipeline specifications for selected eligible cases covering all five stages.
- [] A one-line statement that the reference frame is a labelled synthetic asset.

Troubleshooting

Symptom	What to do
score_needs.py reports a KeyError on a column name	The CSV header must be unedited. Re-copy data/candidate-use-cases.csv from the lab folder; do not open and re-save it in a spreadsheet that renames columns.
Five or six cases pass the threshold	Compare the hand scores with the baseline; investigate the evidence, including the image_supply narrative if present. For this classroom rubric, a case with no repeatable image supply scores at most 2 on repeatability.
No case passes the threshold	Check that you inverted privacy_risk and latency_class as instructed - a high score means the constraint is EASY, not that the risk is high.
You cannot decide a Stage 3 measurement	Example measurements include: a histogram distance, an edge density, a keypoint match count, a detector precision/recall, or a PSNR/SSIM value. Other valid measures include latency, cost and temporal stability. Pick a measure matched to the risk and explain its limitations.

References used in this lab

- [S3] Competency Mapping.docx (TSC ICT-DIT-4022-1.1) - *Course Mapping section and ELO/A-code matrix*.

Verbatim K1-K13 and A1-A8 statements; ELO-to-ability matrix (ELO1:A1 / ELO2:A2 / ELO3:A3,A4 / ELO4:A5 / ELO5:A6 / ELO6:A7,A8).

- [S5] Tertiary Courses public registration page -
<https://www.tertiarycourses.com.sg/wsqsq-generative-ai-for-image-and-video-creation.html>.

Registered public title, TGS code, five delivery topics, six learning outcomes, 2-day/16-hour duration, entry requirements and target job roles.

- [S6] M. Tschochohei and F. Schenker, 'Multimodal Generative AI in the Enterprise: From Pixels to Profit', Packt, June 2026, ISBN 978-1-80611-167-1 - *Ch.5 pp.56-62*.

VAE encoder/decoder and latent space; the reparameterisation trick; GAN generator/discriminator zero-sum game and mode collapse; diffusion forward (fixed Markov chain) and reverse (learned noise prediction) processes; Table 5.1 architecture comparison across image quality, sample diversity, training stability, inference speed and primary use case.

- [S14] A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 - *Ch.1 pp.1-11*.

Historical evolution of synthetic media; role of data in training generative models; overview of major architectures; text-to-image, style transfer and super-resolution as distinct image-synthesis tasks; frame-based versus temporal video generation, motion transfer, and the temporal-consistency and realism challenges.

- [S15] Mewada et al. (2026), as above - *Ch.3 pp.35-44 (Vishnoi, Sisodia, Khare and Yadav, 'Sensors-to-synthesis: Edge AI and IoT for Generative Visual Systems')*.

The four-layer sensors-to-synthesis architecture (IoT sensing / edge intelligence / generative visual modelling / application) shown in Fig. 3.1; MQTT and CoAP as the lightweight sensor-to-edge protocols; split computing between edge and cloud; federated learning; Table 3.2 comparing VAE, GAN and diffusion for edge suitability; Table 3.3 domain applications; Table 3.4 challenges and future directions (compute/energy, latency, privacy/security, scalability/interoperability, ethics; pruning, quantisation, knowledge distillation, neuromorphic and FPGA accelerators, differential privacy, standardised APIs, containerised deployment).

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.